

Fitting the lifetimes of strange particles by reconstructing decays from proton-antiproton collisions at the CDF experiment

Berhan Merha-Tsidk

kebl8364, CO03

14th April 2026

Abstract

In this report, I used data from the CDF collaboration to reconstruct K-short mesons and lambda baryons created in proton-antiproton collisions from the tracks their decay products make. Using information from these tracks I was able to estimate the masses of these two particles. I then estimated the mean lifetimes of these particles by fitting a sample of potential strange particles to an exponential distribution. My result for the lifetime of the K-short was $\tau_{K_S^0} = 1.5_{-0.3}^{+0.5} \times 10^{-10}$ s, differing from the expected result of $\tau_{K_S^0} = (0.8954 \pm 0.0004) \times 10^{-10}$ s by two standard deviations. My result for the lifetime of the lambda baryon was $\tau_{\Lambda^0} = (2.1 \pm 0.2) \times 10^{-10}$ s, differing from the expected result of $\tau_{\Lambda^0} = (2.63 \pm 0.02) \times 10^{-10}$ s by two and a half standard deviations. The source of this error was mostly due to insufficient background cuts, but future experiments could improve on this.

1 Introduction

In December of 1947, G.D. Rochester and C.C. Butler discovered the neutral kaon, produced from cosmic ray events [1]. Specifically, they photographed the decay of the kaon into two oppositely charged pions within a cloud chamber:

$$K^0 \rightarrow \pi^+ + \pi^-. \quad (1.1)$$

In October of 1950, V.D. Hopper and S. Biswas discovered a similar particle from the same conditions [2]. However, this particle had a proton as one of its decay products, distinguishing it as a baryon (rather than a meson like the kaon). This particle was called the lambda baryon:

$$\Lambda^0 \rightarrow p^+ + \pi^-. \quad (1.2)$$

What these particles shared in common is that they were produced copiously (on a timescale of about 10^{-23} seconds) but decayed relatively slowly for their large mass (around 10^{-10} seconds) [3]. These particles were usually produced in pairs, which indicated that there was a property (called “*strangeness*” by Murray Gell-Mann) that was conserved during the particles creation (through the strong force) but not during decay (via the weak force). We now know that this property is caused by the presence of a strange quark in the composition of these particles (K^0 : $d\bar{s}$, Λ^0 : uds).

In 1964 neutral kaons were also discovered to oscillate between it and its antiparticle, which meant that an initial beam of neutral kaons could be measured as antikaons later on. Instead of matter/antimatter eigenstates, kaons travel as the weak (or mass) eigenstates K-long (K_L) and K-short (K_S), with mean lifetimes of 10^{-8} and 10^{-11} seconds respectively. This oscillation was found

to violate CP symmetry in 1964 [4], meaning it slightly prefers decaying into matter particles rather than antimatter. This makes it (partially) the reason why matter dominates over antimatter in our observable universe.*

The Collider Detector at Fermilab (CDF) experiment studies collisions between protons and antiprotons at the Tevatron, which was the world's largest particle accelerator before the Large Hadron Collider (LHC) was built. The goal of the experiment was to “measure exceptional events out of the billions of collisions to [...] look for the production of new physics.” [5] Its most important discovery was that of the top quark in February of 1995 [6], which was the final quark to be discovered in the Standard Model.

In this report I will be using the data collected by this experiment to reconstruct strange particles from their decay products. Using this, I will firstly measure their masses and then finally their lifetimes. The reason we will measure their masses first will be to highlight their “strangeness”, and this will also be a way to test our analyses methods. I will be using object oriented programming in MATLAB for simplicity of use, since we will be applying the same methods (functions) for different sets of data.

2 Methods

The file containing CDF data (named `cdf.dat`) is structured with lines containing information about the event (run number, event number, primary interaction vertex, number of tracks) followed by lines containing information about the tracks measured by the detector (polar angle, curvature, transverse impact parameter and angle, position along beamline). To be able to handle this data, the `CdfDataFile` class makes it easy to extract all the events from the data file when given a CDF data file using the `next()` method. These events are formatted as objects of the class `CdfEvent`, while the tracks themselves are arrays with objects of the class `CdfTrack`.

Tracks in a particle collider are formed due to the fact that charged particles travelling in a magnetic field will form a helical path. This is due to the Lorentz force law:

$$\mathbf{F} = q\mathbf{E} + q\mathbf{v} \times \mathbf{B}, \quad (2.1)$$

where $\mathbf{E}$, $\mathbf{B}$, $\mathbf{v}$ and q are the external electric and magnetic fields and the velocity and charge of the particle respectively. In a collider, detectors provide a magnetic field parallel to the beamline:

$$F = qv_t B, \quad (2.2)$$

where v_t is the transverse velocity of the particle and $\mathbf{F} \perp \mathbf{v}$, $\mathbf{F} \perp \mathbf{B}$. This centripetal force, in conjunction with the perpendicular velocity, results in a helical path which depends on the particles momentum. We can therefore use information about the helical tracks formed to infer more about the event.

`CdfTrack` contains five of these properties:

- `cotTheta` ($\cot \theta$) where θ is the polar angle of track at closest approach to the origin of the CDF coordinate system.
- `curvature` (h) is actually half of the actual track curvature in the transverse plane. It has units of cm^{-1} .
- `d0` (d_0) is the transverse impact parameter at closest approach. It has units of cm.
- `phi0` (ϕ_0) is the angle of the transverse momentum at closest approach. It has units of radians.
- `z0` (z_0) is the position along the beamline at closest approach. It has units of cm.

*CP violation in neutral kaon oscillation is insufficient to fully explain baryonic matter-antimatter asymmetry in the universe.

We can use these properties to model the tracks of particles using the Helix class. This class takes an array of CdfTrack objects and can find information about their tracks through different methods. Specifically, the Helix class finds radius and centre of curvature, transverse momentum and momentum vector, coordinates of the particle over time and the impact parameter of the charged particle.

However, the most important method is the intersect() method. This method takes two helices and finds if and where they intersect. It does this using the transverse circles formed by the charged particles helical paths. If these circles do not intersect, the method gives an empty array. If these circles do intersect, the method solves for the transverse coordinates (of which there are two, denoted w_+ and w_-) and finds the corresponding z coordinate for each helix (denoted as z_1 and z_2). The output is a 4×2 array formatted as follows:

$$\begin{pmatrix} w_{+x} & w_{-x} \\ w_{+y} & w_{-y} \\ \frac{z_{+1}+z_{+2}}{2} & \frac{z_{-1}+z_{-2}}{2} \\ z_{+1} - z_{+2} & z_{-1} - z_{-2} \end{pmatrix}, \quad (2.3)$$

where the subscripts x and y denote which coordinate it is and subscripts $+$ and $-$ denote what solution the z coordinate belongs to. The importance of the fourth row is that, when it comes to governing what the real intersection is, the actual intersection will have a smaller difference in z coordinates.

This method is especially important to us, as it indicates where neutral particles (such as kaons and lambda baryons) have decayed into charged products and, in our analysis, we will use these vertices to find the masses and lifetimes of these particles.

To do this we first create the Vertex class to find information about the decaying particle. This class takes two Helix objects and stores their intersection (an empty set if the two helices do not intersect) as a property assigned to the object. The vertex class has methods for the original particle's momentum, impact parameter and flight distance (all in the transverse plane), which we will later use in analysis to remove background vertices. However, the most important methods find the mass and lifetime of the original particle.

Firstly, the method mass() takes two particles' masses and the vertex object as an input. The particles involved in this decay are moving relativistically, and so to find the mass we must use the energy-momentum relation:

$$E^2 = p^2 + m^2, \quad (2.4)$$

where E , p and m are the respective energy, momentum and mass of a particle ($c = 1$). We assume that we are dealing with a two body decay and apply this relation to each particle involved, along with the conservation of energy and momentum:

$$\begin{aligned} M^2 &= E^2 - P^2 \\ &= (E_1 + E_2)^2 - |\mathbf{p}_1 + \mathbf{p}_2|^2 \\ &= \left(\sqrt{m_1^2 + p_1^2} + \sqrt{m_2^2 + p_2^2} \right)^2 - |\mathbf{p}_1 + \mathbf{p}_2|^2. \end{aligned} \quad (2.5)$$

M , m_1 and m_2 are the masses of the initial and resulting particles respectively and $\mathbf{p}_1$ and $\mathbf{p}_2$ are the momenta of the resulting particles respectively. We then take the square root of this final expression to find the mass of the original particle.

Secondly, the method Lifetime() takes the mass of the original particle, its flight distance and its transverse momentum as inputs to find the lifetime of the particle in its frame:

$$ct = \frac{mL_{xy}}{p_T}, \quad (2.6)$$

where L_{xy} is the flight distance, m is the mass and p_T is the transverse momentum of the particle.

In this report our focus will be on the K-short and the lambda baryon. To first find their masses I made two analysis classes for each particle. The class would take all the track information from each event and find possible intersections that could result from a particle decay. After removing background (more details on this are found in the next section) the remaining vertices would then be assigned a mass using the `mass()` method. To find particle lifetimes a similar method is used, however we now assume the particles mass is a given value, rather than calculating it from our previous analyses.

3 Results and Analysis

The first analysis I did was to find the K-short's mass. The analysis class for this was named `K0SAnalysis`. Firstly, the class only accepted intersections of helices from particles of opposite charge, since we were assuming the K-short only decays into oppositely charged pions (see 1.1). It did this by checking the radii of each helix, which had the charge of the particle encoded in its sign (`radius() = sign(q)|radius|`). The first background cut this class did was on the tracks themselves and tracks with a transverse momentum smaller than 1 GeV/c or an impact parameter smaller than 0.3 cm were removed. The class then did a background cut on the remaining vertices and vertices with a flight distance smaller than 2 cm or an impact parameter greater than 0.5cm were removed.

The class executed all of this in a method named `event()`, which was looped (using another class named `Loop`) to analyse every event and collect results in a property named `massarray`, an array of different measured masses. This was filled into an object of the `Histogram` class called `mass`, using the `fill()` method. You could then plot these results on a histogram using the `plot()` method. The `mass` object was initialised in the `start()` method for the `K0SAnalysis` class and I chose the range of the histogram to be from 0.4 to 0.6 GeV/c². Using the cuts chosen in the lab script [7] gave Fig. 1a but to improve on the signal-to-background ratio I altered the cuts to the tracks' impact parameter and vertices' flight distance. To do this I set these parameters as public properties in the analysis class. Fig. 1b has the minimum flight distance set to 8 cm and the minimum impact parameter set to 1.2 cm. Although this greatly increased the signal-to-background ratio, it did significantly reduce the number of vertices at the peak.

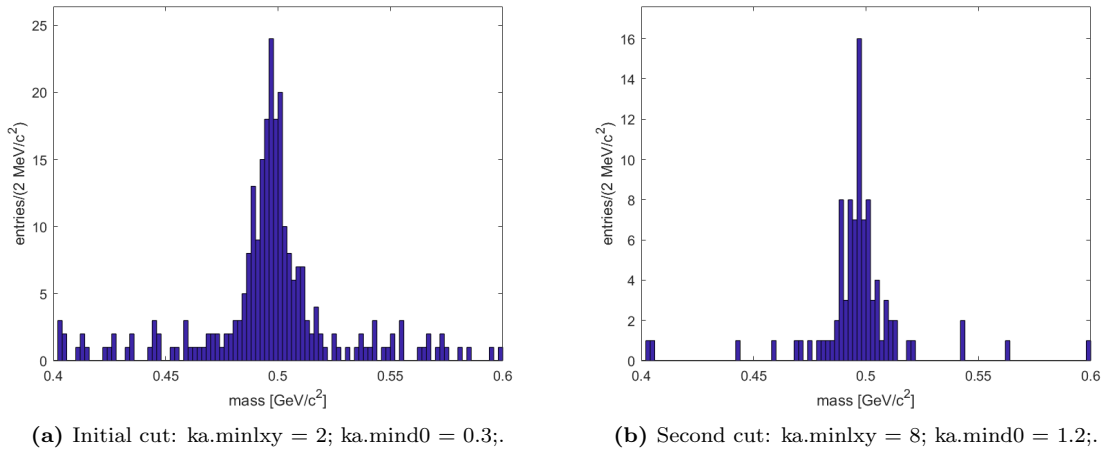


Figure 1: Graph of vertex masses from 0.4 - 0.6 GeV/c² against frequency of measurement for different background cuts. Each bin has a width of 2 MeV/c². `ka.minlxy` is the minimum accepted flight distance for a vertex and `ka.mind0` is the minimum accepted impact parameter for a resulting track.

Both graphs consist of a peak over a uniform background distribution (although Fig. 1b has considerably less background) and this background is mainly due to random tracks being paired together accidentally and passing the background cuts. Both peaks are located at 0.496 - 0.498 GeV/c² which matches the world average K⁰ mass of 0.497614 ± 0.000024 GeV/c² [8]. Furthermore,

the peak has a width of around 20 MeV/c² which I mainly attribute to the fact that we are assuming only one of many decay modes the K_S⁰ can decay through. Table 1 displays the most likely decay modes for a K-short. The most likely decay is one with two oppositely charged pions, which is what we assume is the case in our analysis. However, we also have the smaller probability of other particles being created, such as photons or electrons, or two oppositely charged pions might not form at all. Although these fractions are small, we are analysing thousands of vertices and so there are enough for errors like this to become statistically significant.

K _S ⁰ Decay Modes	
Mode	Branching fraction
$\pi^+ \pi^-$	0.6920 ± 0.0005
$\pi^0 \pi^0$	0.3069 ± 0.0005
$\pi^+ \pi^- \gamma$	$(1.79 \pm 0.05) \times 10^{-3}$
$\pi^\pm e^\mp \nu_e$	$(7.04 \pm 0.08) \times 10^{-4}$
$\pi^+ \pi^- e^+ e^-$	$(4.79 \pm 0.15) \times 10^{-5}$

Table 1: Table of the most prominent decay modes for the K-short. The branching fraction is a measure of the probability of a certain decay mode, given there is a decay, with a maximum possible value of 1. Modes with a branching fraction less than 10^{-5} are not included. [8]

The second analysis was on the lambda baryon's mass. The analysis class for this was named LambdaAnalysis. This class is similar to K0SAnalysis, but with different requirements. In this analysis we assumed all decays of the lambda baryon resulted in a proton and negative pion (see 1.2). This meant the class only accepted intersections of helices with oppositely signed radii, as before. The mass() method depends on two different masses being assigned to each track and so the track with a higher transverse momentum was assigned the proton's mass. Different background cuts were performed: proton tracks with a transverse momentum smaller than 2 GeV/c and tracks with an impact parameter smaller than 0.2 cm were removed.

The LambdaAnalysis class had the same overall structure as K0SAnalysis, except I now chose the range of the histogram to be from 1.0 to 1.2 GeV/c². The background cut I described is from the lab script [7] which gave Fig. 2. I tried various other background cuts, as I did previously, however this time the initial cut seemed to be best to reduce background.

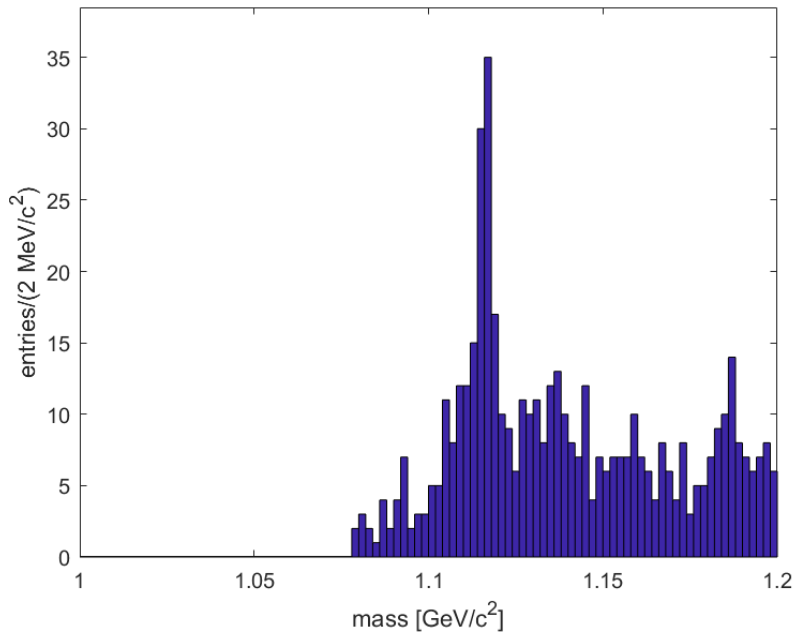


Figure 2: Graph of vertex masses from 1.0 - 1.2 GeV/c² against frequency of measurement for la.minlxy = 2; la.mind0 = 0.2;. Each bin has a width of 2 MeV/c². la.minlxy is the minimum accepted flight distance for a vertex and la.mind0 is the minimum accepted impact parameter for a resulting track.

This graph has a sharper peak over a much larger uniform background distribution for masses above a certain value. The increased background is due to the fact we have less cuts and these cuts are more lenient on the tracks ($\text{la.mind0} < \text{ka.mind0}$). The uniform background distribution also has no values below $1.078 \text{ GeV}/c^2$. This is because conservation of energy must hold in the lambda baryon's rest frame, meaning the mass of the lambda baryon must be greater than or equal to the sum of the masses of the proton and pion ($\sim 1.0778 \text{ GeV}/c^2$). The peak is located at $1.116 - 1.118 \text{ GeV}/c^2$ and has a width of around $10 \text{ MeV}/c^2$. The world average Λ^0 mass is $1.115683 \pm 0.000006 \text{ GeV}/c^2$ [9], which coincides with the peak, but not its centre. This is most likely due to the asymmetry of background; the world average mass is close to the minimum mass possible, and so there is more background events with a larger mass than this world average, shifting the peak higher. The width of the peak is also due to other decay modes, not unlike the K-short (see Table 2). All other decay modes that can affect results (ones with charged particles) have a lower total branching fraction than that of the K-short, which explains why this peak is narrower.

Λ^0 Decay Modes	
Mode	Branching fraction
$p^+ \pi^-$	0.639 ± 0.005
$n \pi^0$	0.358 ± 0.005
$n \gamma$	$(1.75 \pm 0.15) \times 10^{-3}$
$p^+ \pi^- \gamma$	$(8.4 \pm 1.4) \times 10^{-4}$
$p^+ e^- \bar{\nu}_e$	$(8.32 \pm 0.14) \times 10^{-4}$
$p^+ \mu^- \bar{\nu}_\mu$	$(1.57 \pm 0.35) \times 10^{-4}$

Table 2: Table of the most prominent decay modes for the lambda baryon. The branching fraction is a measure of the probability of a certain decay mode, given there is a decay, with a maximum possible value of 1. Modes with a branching fraction less than 10^{-5} are not included. [9]

My final analyses were on these particles' lifetimes. To find the K-short's lifetime, I sampled the vertices within the central peak of the histogram in Fig. 1 using the Lifetime() method, as these were most likely to correspond to actual K-short particles. The cuts on vertex flight distance and track impact parameter will introduce an unwanted bias to the analysis of particle lifetime, so we get rid of these cuts, significantly increasing background in the process. To get rid of background we sample the lifetimes from vertices in bands adjacent to the peak, find the average, and subtract this from our initial sample. For the kaon, the initial sample is on vertices with mass between 0.48 and $0.52 \text{ GeV}/c^2$ and the background sample is in the range $0.46 - 0.48 \text{ GeV}/c^2$ and $0.52 - 0.54 \text{ GeV}/c^2$.

Once this data was obtained I had to check how well it fit the expected data. A sample of lifetimes of decaying particles will follow an exponential distribution:

$$y(t; A, \tau) = \frac{A}{c\tau} e^{-ct/c\tau}, \quad (3.1)$$

where t is the lifetime, τ is the mean lifetime and A is a dimensionless constant such that the integral of y from 0 to ∞ is A . To see how well the data fit I performed a chi-squared test:

$$\chi^2 = \sum_i \frac{(y_i - \frac{A}{c\tau} e^{-ct_i/c\tau})^2}{\sigma_i^2}, \quad (3.2)$$

where y_i is taken from our sample, σ is the uncertainty in each measurement and we are summing over each discrete step in time in which a certain number of particles have decayed. I calculated this uncertainty by simply finding the square root of the number of decaying particles at each time step (assuming both samples follow a Poisson distribution and then adding errors in quadrature). Varying the test for different values of A and $c\tau$, I made a surface plot to find the minimum chi-squared value (Fig. 3a).

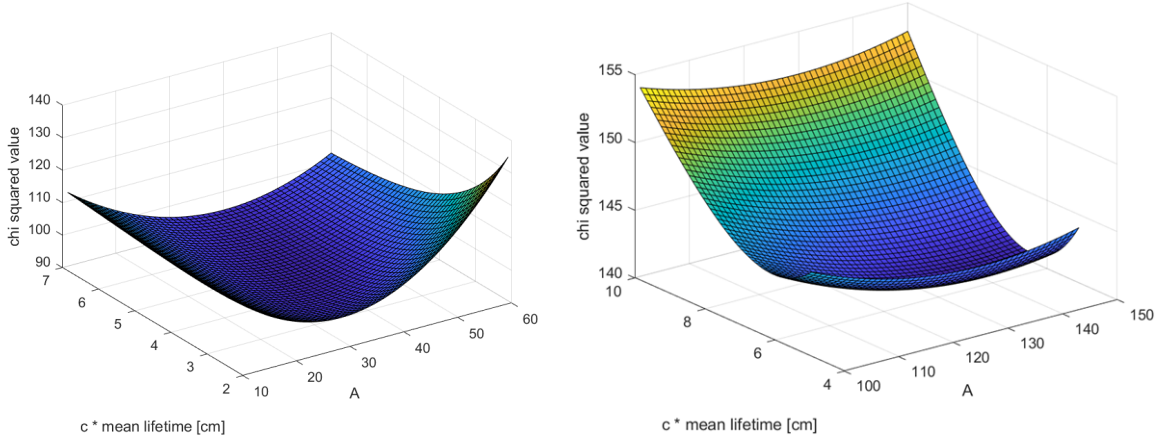
Using Fig. 3a, I found the minimum chi-squared value to be $\chi_{min}^2 = 92.3153$, for the values $A = 36$ and $c\tau = 4.5 \text{ cm}$. There were around 100 degrees of freedom here, which indicated an extremely good

fit because $\frac{\chi_{min}^2}{\text{degrees of freedom}} < 1$. I also estimated the standard deviation in $c\tau$ by seeing looking for the value where $\chi^2 > \chi_{min}^2 + 1$. This gave me a final result of:

$$\tau_{K_S^0} = 1.5_{-0.3}^{+0.5} \times 10^{-10} \text{ s.} \quad (3.3)$$

The expected result was $\tau_{K_S^0} = (0.8954 \pm 0.0004) \times 10^{-10} \text{ s}$ [10], lying around two standard deviations below my result. The reason my result is so high is most likely due to the background cut not being good enough, leaving enough background results to skew the final result. At a closer look at the data, many vertices that passed the cut seemed to live abnormally long, which further confirmed this hypothesis.

When applying a similar analysis to the lambda baryon, I obtained another surface plot to fit the lifetime (Fig. 3b).



(a) K-short meson. $c\tau$ takes values between 2 and 7 cm, (b) Lambda baryon. $c\tau$ takes values between 4 and 10 cm, corresponding to lifetimes from $0.7 - 2.3 \times 10^{-10} \text{ s}$. corresponding to lifetimes from $1.3 - 3.3 \times 10^{-10} \text{ s}$.

Figure 3: Graph of χ^2 value for different exponential distributions for each particle: $y(t; A, \tau) = \frac{A}{c\tau} e^{-ct/c\tau}$.

This plot gave me a minimum chi-squared value of $\chi_{min}^2 \sim 140$ which suggested less of a good fit, as there was the same number of degrees of freedom. At this minimum, $c\tau = 6.4 \text{ cm}$, which gave me a final result of:

$$\tau_{\Lambda^0} = (2.1 \pm 0.2) \times 10^{-10} \text{ s.} \quad (3.4)$$

The expected result was $\tau_{\Lambda^0} = (2.63 \pm 0.02) \times 10^{-10} \text{ s}$ [11], lying two and a half standard deviations above my result. My result was now too low for a similar reason, a lot of the background now consisted of short-lasting particles, which also could not be fully removed.

The reason background vertices have an opposite effect for each particle is because their lifetimes differ by almost an order of magnitude. Because the background is mostly randomly paired, with many different lifetimes, more of the background will have lifetimes shorter than that of the lambda baryon and longer than that of the K-short, because the lambda baryon is very long-lived, while the K-short is very short-lived (in comparison to each other).

4 Conclusion

Overall, this report aimed to highlight the importance of strange particle and their properties. One important property we managed to showcase is the longevity of strange particles for their mass. For example, we found the K-short to have a mass of around $500 \text{ MeV}/c^2$ and a lifetime on the order of 10^{-10} s . A non-strange particle with a much smaller mass is the pion, with a mass of around $100 \text{ MeV}/c^2$, but its lifetime is either around the same order as the K-short ($\pi^\pm$) or much shorter ($\tau \sim 10^{-17} \text{ s}$ for the π^0). Similarly, we found the lambda baryon to have a mass on the order of 1

GeV/c² and a lifetime on the order of 10⁻¹⁰ s. The delta baryon, with a similar mass, has a lifetime on the order of 10⁻²⁴ s.

The main source of error in this report is background vertices accidentally passing through our efforts to cut them out. The main solution to this problem would most likely be to use more advanced techniques to cut more background. Using machine learning techniques, given background data for training, could be the next step forward in refining our results.

References

- [1] G. D. Rochester and C. C. Butler. “Evidence for the Existence of New Unstable Elementary Particles”. In: *Nature* 160 (1947).
- [2] V. D. Hopper and S. Biswas. “Evidence Concerning the Existence of the New Unstable Elementary Neutral Particle”. In: *Phys. Rev.* 80 (6 Dec. 1950), pp. 1099–1100. URL: <https://link.aps.org/doi/10.1103/PhysRev.80.1099>.
- [3] David Griffiths. *Introduction to Elementary Particles*. Wiley-VCH, 2008.
- [4] J. H. Christenson et al. “Evidence for the 2 π Decay of the K_2^0 Meson”. In: *Phys. Rev. Lett.* 13 (4 July 1964), pp. 138–140. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.13.138>.
- [5] John Yoh. Archived on 4th December 2004, page cannot be accessed as of 4th April 2026. Nov. 2004. URL: <https://web.archive.org/web/20041204192055/http://www-cdf.fnal.gov:80/events/cdfintro.html>.
- [6] F. Abe et al. “Observation of Top Quark Production in $\bar{p}p$ Collisions with the Collider Detector at Fermilab”. In: *Physical Review Letters* 74.14 (Apr. 1995), pp. 2626–2631. URL: <http://dx.doi.org/10.1103/PhysRevLett.74.2626>.
- [7] University of Oxford. *CO03 - Strange mesons from proton-antiproton collisions at the CDF experiment*. Published with MATLAB® R2013a. URL: <https://www-teaching.physics.ox.ac.uk/CO-MATLAB/html/co03/co03.html>.
- [8] K.A. Olive et al. (Particle Data Group). *STRANGE MESONS*. 2014. URL: <https://pdg.lbl.gov/2014/tables/rpp2014-tab-mesons-strange.pdf>.
- [9] P.A. Zyla et al. (Particle Data Group). *Λ BARYONS*. 2020. URL: <https://pdg.lbl.gov/2020/tables/rpp2020-tab-baryons-Lambda.pdf>.
- [10] P.A. Zyla et al. (Particle Data Group). *Particle listings – K_S^0* . 2020. URL: <https://pdg.lbl.gov/2021/listings/rpp2021-list-K-zero-S.pdf>.
- [11] C. Amsler et al. (Particle Data Group). *Λ* . 2008. URL: <https://pdg.lbl.gov/2008/listings/s018.pdf>.

Appendices

A Code for the classes

CdfDataFile.m:

```

1 classdef CdfDataFile < handle
2     % CdfDataFile wraps CDF track data file.
3     % This class reads a CDF track data file sequentially.
4     % The format:  each event consists of the following:
5     %   line 1:  runNumber eventNumber vertex-x vertex-y numTracks
6     %   runNumber = CDF run number
7     %   eventNumber = CDF event number within run
8     %   vertex-x, vertex-y = transverse location (xy) of primary
        interaction point

```

```

9      %      numTracks = number of reconstructed tracks
10     %      lines 2 to numTracks-2:  cotTheta curvature d0 phi0 z0
11     %      cotTheta = polar angle of track at closest approach to (0,0)
12     %      curvature = track curvature (actually 0.5*curvature) in xy
13     %      plane
14     %      d0 = impact parameter with respect to origin at point of
15     %      closest approach
16     %      phi0 = azimuthal angle at point of closest approach
17     %      z0 = position along beamline at point of closest approach
18
19     properties(Access=protected)
20         id; % file identifier
21     end
22
23     methods
24         % constructor:
25         function obj = CdfDataFile(filename)
26             obj.id = -1; % invalid
27             if ischar(filename)
28                 % string -> filename
29                 obj.id = fopen(filename);
30             elseif isinteger(filename)
31                 % integer -> file identifier
32                 obj.id = filename;
33             end
34         end
35
36         % return true if file is open (and valid)
37         function result = isOpen(obj)
38             result = (obj.id > 0);
39         end
40
41         % close file
42         function close(obj)
43             if obj.isOpen()
44                 fclose(obj.id);
45                 obj.id = -1;
46             end
47         end
48
49         % start reading again from the beginning
50         function rewind(obj)
51             if obj.isOpen()
52                 frewind(obj.id);
53             end
54         end
55
56         % get next event
57         function event = next(obj)
58             event = CdfEvent(); % created in invalid state
59             if obj.isOpen()
60                 while 1

```

```

60         str = fgets(obj.id);
61         if str == -1
62             return % end of file - leaves event in invalid
                state
63         end
64         s = strtrim(str); % trim the leading/ending
                whitespace
65         if numel(s) <= 0
66             continue % usually just an end of line
67         end
68         if s(1) ~= '#'
69             break; % get on with event
70         else
71             disp(s);
72         end
73     end
74     % parse event header
75     expected = 5;
76     [head, n] = sscanf(s, '%d %d %g %g %d', 5);
77     if n == expected
78         % read tracks
79         expected = 5 * head(5);
80         [pars, n] = fscanf(obj.id, '%g', [5, head(5)]);
81         if n ~= expected
82             return % unexpected end of track data
83         end
84     else % unexpected end of header
85         return % leaves event in invalid state
86     end
87     event.runNumber = head(1);
88     event.eventNumber = head(2);
89     event.vertex = head(3:4);
90     event.tracks = CdfTrack(pars);
91 end
92 end
93
94 end % methods
95
96 end % classdef

```

CdfEvent.m:

```

1  classdef CdfEvent < handle
2      % CdfEvent contains CDF event data
3
4      properties(SetAccess=?CdfDataFile)
5          runNumber      % integer:  CDF run number
6          eventNumber    % integer:  CDF event number within run
7          vertex         % 2x1 float:  transverse location (xy) of primary
                interaction vertex
8          tracks         % array of CdfTrack:  track parameters
9      end
10
11      methods

```

```

12
13     % constructor:  create CdfEvent in invalid state
14     function obj = CdfEvent()
15         obj.runNumber = -1;
16         obj.eventNumber = -1;
17         obj.vertex = [0; 0];
18         obj.tracks = [];
19     end
20
21     % test if this event is valid
22     function valid = isValid(obj)
23         valid = (obj.runNumber >= 0);
24     end
25
26     end % methods
27
28 end % classdef

```

CdfTrack.m:

```

1 classdef CdfTrack
2     % CdfTrack contains CDF track parameters
3
4     properties(SetAccess=protected)
5         cotTheta    % polar angle of track at closest approach to (0,0)
6         curvature    % track curvature (actually 0.5*curvature) in xy
           plane (1/cm)
7         d0          % transverse impact parameter at closest approach (
           cm)
8         phi0        % angle in transverse plane at closest approach (
           radians)
9         z0          % position along beamline at closest approach (cm)
10    end
11
12    methods
13
14        % constructor:  accepts a 5xN matrix of track parameters
15        function obj = CdfTrack(v)
16            if nargin > 0
17                n = size(v,2); % number of tracks
18                obj(n) = CdfTrack; % preallocate array of objects
19
20                if (size(v,1) == 5)
21                    for i = 1:n
22                        obj(i).cotTheta = v(1,i);
23                        obj(i).curvature = v(2,i);
24                        obj(i).d0 = v(3,i);
25                        obj(i).phi0 = v(4,i);
26                        obj(i).z0 = v(5,i);
27                    end
28                else
29                    % invalid inputs, so create array of invalid
                       tracks
30                    for i = 1:n

```

```

31         obj(i) = CdfTrack();
32     end
33 end
34 else
35     % no arguments - create an invalid CdfTrack
36     obj.cotTheta = -999.0;
37     obj.curvature = -999.0;
38     obj.d0 = -999.0;
39     obj.phi0 = -999.0;
40     obj.z0 = -999.0;
41 end
42 end
43
44 % check if this object is valid
45 function valid = isValid(obj)
46     valid = (obj.cotTheta == -999.0);
47 end
48
49 end % methods
50 end % classdef

```

Helix.m:

```

1 classdef Helix
2     % Helix contains data about path of helix traced by particles:
3     % 'radius' is the radius of curvature of the helix path
4     % 'centre' is the centre of curvature of the helix path
5
6     properties(Access=?Vertex)
7         trk;
8     end
9     properties(Constant)
10         kpc = 0.002116;
11     end
12
13     methods
14
15         % constructor
16         function obj = Helix(t)
17             if nargin > 0
18                 obj(numel(t)) = Helix; % create an array of the right
19                                         size
20                 for i = 1:numel(t)
21                     obj(i).trk = t(i);
22                 end
23             end
24
25             % finds radius of curvature
26             function radius = radius(t)
27                 radius = 1/(2*t.trk.curvature);
28             end
29
30             % finds centre of curvature in x-y plane

```

```

31 function centre = centre(t)
32     r = radius(t);
33     d0 = t.trk.d0;
34     phi = t.trk.phi0;
35     wc = (r+d0)*exp(1i*(phi+pi/2)); % centre encoded as
        complex exponential
36     centre(1) = real(wc); % real part is the x coord
37     centre(2) = imag(wc); % imag part is the y coord
38 end
39
40 % finds points on the particles track over time
41 function p = points(tr,t) % tr is track, t is time
42     r = radius(tr);
43     d0 = tr.trk.d0;
44     phi0 = tr.trk.phi0;
45     z0 = tr.trk.z0;
46     cotTheta = tr.trk.cotTheta;
47     q = sign(radius(tr)); % particle's charge depends on the '
        sign' of its radius
48     wc = (r+d0)*exp(1i*(phi0+pi/2));
49     w = wc - 1i*r*exp(1i*(phi0+q*t)); % position in x-y plane
        (complex)
50     p(1,:) = real(w);
51     p(2,:) = imag(w);
52     p(3,:) = z0 + abs(r)*t*cotTheta; % z position
53 end
54
55 % finds intersection of two helices
56 function w = intersect(tr,h)
57     if ~isa(h, 'Helix')
58         disp('Argument of intersect() is not a Helix');
59         w = []; % no intersection
60         return;
61     end
62     c1 = tr.centre();
63     c2 = h.centre();
64     d = c2 - c1;
65     d2 = d(1)*d(1) + d(2)*d(2); % square of distance between
        centers
66     r1 = tr.radius();
67     r2 = h.radius();
68     if (d2 > (abs(r1)+abs(r2))^2)
69         w = []; % no intersection
70         return;
71     end
72     alpha = acos((norm(c2-c1)^2 + r1^2 - r2^2) / (2 * norm(c2-
        c1) * abs(r1))); % equations given in lab script
73     delta = atan2(c2(2) - c1(2), c2(1) - c1(1));
74     w = zeros([4,2]); % initialising
75     w(1,1) = c1(1) + abs(r1) * cos(delta + alpha);
76     w(1,2) = c1(1) + abs(r1) * cos(delta - alpha);
77     w(2,1) = c1(2) + abs(r1) * sin(delta + alpha);
78     w(2,2) = c1(2) + abs(r1) * sin(delta - alpha);

```

```

79         q1 = sign(r1);
80         q2 = sign(r2);
81         t1p = (asin( (w(1,1) - c1(1)) / r1 ) - tr.trk.phi0) / q1;
            % solved eqs for points()
82         t2p = (asin( (w(1,1) - c2(1)) / r2 ) - h.trk.phi0) / q2;
83         t1m = (asin( (w(1,2) - c1(1)) / r1 ) - tr.trk.phi0) / q1;
84         t2m = (asin( (w(1,2) - c2(1)) / r2 ) - h.trk.phi0) / q2;
85         v1p = tr.points(t1p); % contains z position at point of
            intersection in (x, y)
86         v2p = h.points(t2p);
87         v1m = tr.points(t1m);
88         v2m = h.points(t2m);
89         w(3,1) = (v1p(3) + v2p(3))/2; % average z position
90         w(3,2) = (v1m(3) + v2m(3))/2;
91         w(4,1) = v1p(3) - v2p(3); % difference in z positions
92         w(4,2) = v1m(3) - v2m(3);
93     end
94
95     % finds transverse (x-y plane) momentum of particle
96     function x = pt(obj)
97         x = obj.kpc / obj.trk.curvature;
98     end
99
100    % finds momentum of particle in x,y and z direction
101    function x = p3(obj, v)
102        h = obj.trk.curvature;
103        d0 = obj.trk.d0;
104        phi0 = obj.trk.phi0;
105        x(1) = abs(obj.pt) * ((1 + 2 * h * d0) * cos(phi0) - 2 * h
            * v(2));
106        x(2) = abs(obj.pt) * ((1 + 2 * h * d0) * sin(phi0) + 2 * h
            * v(1));
107        x(3) = abs(obj.pt) * obj.trk.cotTheta;
108    end
109
110    % finds impact parameter of particle
111    function dPV = ip(obj, wPV) % wPV is primary interaction
        vertex (proton-antiproton collision location)
112        wc = obj.centre;
113        dPV = abs(wc(1) + wc(2)*1i - wPV(1) - wPV(2)*1i) - abs(obj
            .radius);
114    end
115
116
117     end % methods
118 end % classdef

```

Vertex.m:

```

1 classdef Vertex
2     % Vertex contains data about a particle which has decayed into two
        new
3     % particles
4     % 'h1' is one of the resulting paths

```

```

5      % 'h2' is the other resulting path
6      % 'vtx' is the coordinate of decay
7
8      properties(Access=protected)
9          h1
10         h2 % initial helices
11     end
12     properties(Access={?K0SAnalysis, ?LambdaAnalysis})
13         vtx % vertex of intersection (point of decay)
14     end
15
16     methods
17
18         % constructor
19         function obj = Vertex(h) % Input must be a 1x2 array of
20             helices that form vertex
21             obj.h1 = h(1);
22             obj.h2 = h(2);
23             obj.vtx = zeros(1,2);
24             v = h(1).intersect(h(2));
25             if isempty(v)
26                 obj.vtx = v;
27             elseif norm(v(4,1)) > norm(v(4,2)) % checking which
28                 intersection seems more accurate by finding the smaller
29                 difference in z values
30                 obj.vtx(1) = v(1,2);
31                 obj.vtx(2) = v(2,2);
32             else
33                 obj.vtx(1) = v(1,1);
34                 obj.vtx(2) = v(2,1);
35             end
36         end
37
38         function pT = pT(obj)
39             p1 = obj.h1.p3(obj.vtx);
40             p2 = obj.h2.p3(obj.vtx); % initial transverse momenta
41             pT = p1(1:2) + p2(1:2); % conservation of momentum
42         end
43
44         % function uses SR to find mass of decaying particle (c = 1 of
45         % course)
46         function x = mass(obj, mass1, mass2)
47             p1 = obj.h1.p3(obj.vtx); % momenta of resulting particles
48             p2 = obj.h2.p3(obj.vtx);
49             E = sqrt(mass1^2 + norm(p1)^2) + sqrt(mass2^2 + norm(p2)
50                 ^2); % finding the energy of the decaying particle
51                 INCLUDING NON TRANSVERSE MOMENTUM
52             x = sqrt(E^2 - norm(p1 + p2)^2); % using the energy
53                 momentum relation to find the mass of the decaying
54                 particle
55         end
56
57         % finds flight distance IN THE TRANSVERSE DIRECTION

```

```

50     function L_xy = FlightDistance(obj, piv)
51         pThat = obj.pT / norm(obj.pT);
52         L_xy = dot(obj.vtx - piv, pThat); % equation for L_xy
53     end
54
55     % finds impact parameter of initial particle
56     function d_0 = ImpactParameter(obj, piv)
57         v = [obj.vtx, 0]; % every array must be 1x3 so we can do a
           cross product
58         wPV = [piv, 0];
59         pThat = [obj.pT, 0] / norm([obj.pT, 0]);
60         d_0v = cross((v - wPV), pThat);
61         d_0 = d_0v(3);
62     end
63
64     function ct = Lifetime(obj, m, piv)
65         ct = abs(obj.FlightDistance(piv) * (m / norm(obj.pT)));
66     end
67
68     end % methods
69 end % classdef

```

Histogram.m:

```

1  classdef Histogram < handle
2      % Histogram accumulates data in pre-defined bins
3
4      properties(SetAccess=protected)
5          nbins; % number of bins
6          xlo; % minimum x
7          xhi; % maximum x
8          data; % bin contents
9          underflow; % underflow counter
10         overflow; % overflow counter
11         dx; % bin width
12     end
13
14     methods
15
16         % constructor
17         function obj = Histogram(nbins, xlo, xhi)
18             obj.nbins = nbins;
19             obj.xlo = xlo;
20             obj.xhi = xhi;
21             obj.data = zeros(1, nbins);
22             obj.underflow = 0;
23             obj.overflow = 0;
24             obj.dx = (xhi - xlo) / nbins;
25         end
26
27         % return bin low-edges vector
28         function b = bins(obj)
29             bb = (obj.xlo : obj.dx : obj.xhi);

```

```

30         b = bb(1:obj.nbins);
31     end
32
33     % accumulate data
34     function fill(obj, v)
35         for index = 1:numel(v)
36             x = v(index);
37             if x < obj.xlo
38                 obj.underflow = obj.underflow + 1;
39             elseif x >= obj.xhi
40                 obj.overflow = obj.overflow + 1;
41             else
42                 ibin = floor((x-obj.xlo)/obj.dx) + 1;
43                 obj.data(ibin) = obj.data(ibin) + 1;
44             end
45         end
46     end
47
48     % return total number of data points accumulated
49     function n = total(obj)
50         n = obj.underflow + sum(obj.data) + obj.overflow;
51     end
52
53     % return maximum bin contents
54     function n = max(obj)
55         n = max(obj.data);
56     end
57
58     % plot the histogram
59     function plot(obj)
60         figure;
61         binranges = obj.bins();
62         bar(binranges, obj.data, 'histc');
63         axis([ obj.xlo obj.xhi 0 obj.max()*1.1 ]);
64     end
65 end
66
67 end

```

KOSAnalysis.m:

```

1  classdef KOSAnalysis < Analysis
2
3      properties (Constant)
4          mp = 0.13957
5          mk = 0.497614
6      end
7      properties (Access=protected)
8          massarray % contains all the masses
9          ctarray % contains all the lifetimes
10         ctsidearray % contains all the background lifetimes
11         lxyarray % contains all the flight distances
12         lxyarray % contains all the background flight distances
13     end

```

```

14 properties (Access=public)
15     mass % contains all masses in Histogram class
16     ct % contains all lifetimes in Histogram class
17     ctside % contains all background lifetimes in Histogram class
18     lxy % contains all flight distances in Histogram class
19     lxyside % contains all background flight distances in
        Histogram class
20     minlxy % condition for flight distance
21     mind0 % condition for impact parameter (of individual tracks)
22 end
23
24 methods
25
26     % constructor
27     function obj = KOSAnalysis()
28         obj.massarray = []; % this will contain all the measured
            masses
29         obj.mass = []; % this will become a Histogram object that
            we can plot
30         obj.ctarray = []; % this will contain all the measured
            lifetimes
31         obj.ct = []; % this will become a Histogram object that we
            can plot
32         obj.ctsidearray = []; % this will contain all the measured
            background lifetimes
33         obj.ctside = []; % this will become a Histogram object
            that we can plot
34         obj.lxyarray = []; % this will contain all the measured
            flight distances
35         obj.lxy = []; % this will become a Histogram object that
            we can plot
36         obj.lxysidearray = []; % this will contain all the
            measured background flight distances
37         obj.lxyside = []; % this will become a Histogram object
            that we can plot
38         obj.mind0 = 0.3;
39         obj.minlxy = 2;
40     end
41
42     % setting up the histogram
43     function start(obj)
44         obj.mass = Histogram(100, 0.4, 0.6); % 100 bins from 0.4
            to 0.6 GeV/c^2
45         obj.ct = Histogram(100, 0, 15); % 100 bins from 0 to 15 cm
46         obj.ctside = Histogram(100, 0, 15);
47         obj.lxy = Histogram(100, 0, 15);
48         obj.lxyside = Histogram(100, 0, 15);
49     end
50
51     % filling the histogram up with the measured masses and
        lifetimes
52     function stop(obj)
53         obj.mass.fill(obj.massarray)

```

```

54         obj.ct.fill(obj.ctarray)
55         obj.ctime.fill(obj.ctimearray)
56         obj.lxy.fill(obj.lxyarray)
57         obj.lxy.fill(obj.lxyarray)
58     end
59
60     % executes every event to find measured masses and lifetimes
61     function event(obj, ev)
62         piv = ev.vertex'; % transposing the primary interaction
63         vertex for convenience (from 2x1 to 1x2)
64         tracks = ev.tracks;
65         allhelices = Helix(tracks); % all helices for an event
66         helices = [];
67         for k = 1:numel(allhelices)
68             if all([abs(allhelices(k).pt) > 1, abs(allhelices(k).
69                 ip(piv)) > obj.mind0]) % finding helices which
70                 match our criteria
71                 helices = [helices, allhelices(k)];
72             end
73         end
74         for i = 2:numel(helices)
75             h1 = helices(i);
76             r1 = h1.radius;
77             for j = 1:i-1 % for all possible combination of
78                 helices...
79                 h2 = helices(j);
80                 r2 = h2.radius;
81                 v = Vertex([h1, h2]);
82                 vertex_ij = v.vtx;
83                 if isempty(vertex_ij) % if no intersection is
84                     found
85                     continue
86                 end
87                 if all([sign(r1) ~= sign(r2), v.FlightDistance(piv
88                     ) > obj.minlxy, abs(v.ImpactParameter(piv)) <
89                     0.5]) % more criteria for our chosen
90                     intersection
91                     obj.massarray = [obj.massarray, v.mass(obj.mp,
92                         obj.mp)]; % assuming both tracks are pions
93                     , we find the predicted mass and lifetime
94                     of the decaying particle
95                     if v.mass(obj.mp, obj.mp) > 0.48 && v.mass(obj
96                         .mp, obj.mp) < 0.52
97                         obj.ctarray = [obj.ctarray, v.Lifetime(obj
98                             .mk, piv)]; % vertices within the peak
99                             are added to lifetime array
100                         obj.lxyarray = [obj.lxyarray, v.
101                             FlightDistance(piv)]; % vertices within
102                             the peak are added to flight distance
103                             array
104                     elseif v.mass(obj.mp, obj.mp) > 0.46 && v.mass
105                         (obj.mp, obj.mp) < 0.48
106                         obj.ctimearray = [obj.ctimearray, v.

```

```

89         Lifetime(obj.mk, piv)]; % vertices just
          outside the peak are measured as
          background
90         obj.lxysidearray = [obj.lxysidearray, v.
          FlightDistance(piv)];
91     elseif v.mass(obj.mp, obj.mp) < 0.54 && v.mass
          (obj.mp, obj.mp) > 0.52
92         obj.ctsidearray = [obj.ctsidearray, v.
          Lifetime(obj.mk, piv)];
          obj.lxysidearray = [obj.lxysidearray, v.
          FlightDistance(piv)];
93     end
94 end
95     end % second loop of helices
96 end % first loop of helices
97 end % event()
98
99 end % methods
100 end % classdef

```

LambdaAnalysis.m:

```

1 classdef LambdaAnalysis < Analysis
2
3     properties (Constant)
4         mpion = 0.13957
5         mproton = 0.938272
6         ml = 1.115683
7     end
8     properties (Access=protected)
9         massarray % contains all the masses
10        ctarray % contains all the lifetimes
11        ctsidearray % contains all the background lifetimes
12    end
13    properties (Access=public)
14        mass % contains all masses in Histogram class
15        ct % contains all lifetimes in Histogram class
16        ctside % contains all background lifetimes in Histogram class
17        minlxy % condition for flight distance
18        mind0 % condition for impact parameter (of individual tracks)
19    end
20
21    methods
22
23        % constructor
24        function obj = LambdaAnalysis()
25            obj.massarray = []; % this will contain all the measured
                masses
26            obj.mass = []; % this will become a Histogram object that
                we can plot
27            obj.ctarray = []; % this will contain all the measured
                lifetimes
28            obj.ct = []; % this will become a Histogram object that we
                can plot

```

```

29         obj.ctimearray = []; % this will contain all the measured
        background lifetimes
30         obj.ctime = []; % this will become a Histogram object
        that we can plot
31         obj.mind0 = 0.2;
32         obj.minlxy = 2;
33     end
34
35     % setting up the histogram
36     function start(obj)
37         obj.mass = Histogram(100, 1.0, 1.2); % 100 bins from 1.0
        to 1.2 GeV/c^2
38         obj.ct = Histogram(100, 0, 30); % 100 bins from 0 to 30 cm
39         obj.ctime = Histogram(100, 0, 30);
40     end
41
42     % filling the histogram up with the measured masses and
        lifetimes
43     function stop(obj)
44         obj.mass.fill(obj.massarray)
45         obj.ct.fill(obj.ctarray)
46         obj.ctime.fill(obj.ctimearray)
47     end
48
49     % executes every event to find measured masses and lifetimes
50     function event(obj, ev)
51         piv = ev.vertex'; % transposing the primary interaction
        vertex for convenience (from 2x1 to 1x2)
52         tracks = ev.tracks;
53         allhelices = Helix(tracks); % all helices for an event
54         helices = [];
55         for k = 1:numel(allhelices)
56             if abs(allhelices(k).ip(piv)) > obj.mind0 % finding
                helices which match our criteria
57                 helices = [helices, allhelices(k)];
58             end
59         end
60         for i = 2:numel(helices)
61             h1 = helices(i);
62             r1 = h1.radius;
63             for j = 1:i-1 % for all possible combination of
                helices...
64                 h2 = helices(j);
65                 r2 = h2.radius;
66                 v = Vertex([h1, h2]);
67                 vertex_ij = v.vtx;
68                 if isempty(vertex_ij) % if no intersection is
                    found
69                     continue
70                 end
71                 if (sign(r1) ~= sign(r2)) && (v.FlightDistance(piv)
                    > obj.minlxy) % more criteria for our chosen
                    intersection

```

```

72     if (h1.pt > h2.pt) && (abs(h1.pt) > 2) %
       finding whichever track has a higher pt and
       assigning it the mass of a proton (also
       must match criteria)
73         obj.massarray = [obj.massarray, v.mass(obj
74             .mproton, obj.mpion)];
       if v.mass(obj.mproton, obj.mpion) > 1.10
       && v.mass(obj.mproton, obj.mpion) <
       1.14
75         obj.ctarray = [obj.ctarray, v.Lifetime
           (obj.ml, piv)]; % vertices within
           the peak are added to lifetime
           array
76     elseif v.mass(obj.mproton, obj.mpion) >
       1.08 && v.mass(obj.mproton, obj.mpion)
       < 1.10
77         obj.ctsidearray = [obj.ctsidearray, v.
           Lifetime(obj.ml, piv)]; % vertices
           just outside the peak are measured
           as background
78     elseif v.mass(obj.mproton, obj.mpion) >
       1.14 && v.mass(obj.mproton, obj.mpion)
       < 1.16
79         obj.ctsidearray = [obj.ctsidearray, v.
           Lifetime(obj.ml, piv)];
80     end
81     elseif (h1.pt < h2.pt) && (abs(h2.pt) > 2)
82         obj.massarray = [obj.massarray, v.mass(obj
83             .mpion, obj.mproton)];
       if v.mass(obj.mpion, obj.mproton) > 1.10
       && v.mass(obj.mpion, obj.mproton) <
       1.14
84         obj.ctarray = [obj.ctarray, v.Lifetime
           (obj.ml, piv)]; % vertices within
           the peak are added to lifetime
           array
85     elseif v.mass(obj.mpion, obj.mproton) >
       1.08 && v.mass(obj.mpion, obj.mproton)
       < 1.10
86         obj.ctsidearray = [obj.ctsidearray, v.
           Lifetime(obj.ml, piv)]; % vertices
           just outside the peak are measured
           as background
87     elseif v.mass(obj.mpion, obj.mproton) >
       1.14 && v.mass(obj.mpion, obj.mproton)
       < 1.16
88         obj.ctsidearray = [obj.ctsidearray, v.
           Lifetime(obj.ml, piv)];
89     end
90     end
91     end
92     end % second loop of helices
93 end % first loop of helices

```

```

94         end % event()
95
96     end
97 end

```

Loop.m:

```

1  classdef Loop < handle
2      %Loop loops over a data file of CDF tracks.
3
4      properties(SetAccess=protected)
5          file; % CdfDataFile
6      end
7      properties % override these public properties
8          nevmax; % number of events to process
9          report; % number of events between each console report
10     end
11
12     methods
13
14         % constructor, using the filename of the data file
15         function obj = Loop(filename)
16             obj.file = CdfDataFile(filename);
17             obj.nevmax = 10000; % (provided cdf.dat has 10000 events
18                                 anyway)
19             obj.report = 10;
20         end
21
22         % close, just in case we need it
23         function close(obj)
24             obj.file.close();
25         end
26
27         % main loop
28         function run(obj, analyses)
29
30             % check if there are multiple analyses to run
31             if (iscell(analyses))
32                 ca = analyses;
33             else
34                 ca = { analyses };
35             end
36
37             % check that all the analyses are indeed of the Analysis
38             class
39             na = numel(ca);
40             for ia = 1:na
41                 if ~isa(ca{ia}, 'Analysis')
42                     fprintf(1, 'Analysis list contains non-Analysis
43                             objects\n');
44                     return;
45                 end
46             end
47         end
48     end
49 end

```

```

44
45     % start at the beginning of the data file
46     obj.file.rewind();
47
48     % call start() on each Analysis
49     for ia = 1:na
50         ca{ia}.start();
51     end
52     fprintf(1, 'Begin analyses\n');
53
54     % read and analyze events one at a time
55     nev = 0; % count events
56     while 1
57         ev = obj.file.next();
58         if ev.isValid()
59             for ia = 1:na
60                 ca{ia}.event(ev);
61             end
62             nev = nev + 1;
63             if mod(nev, obj.report) == 0
64                 fprintf(1, '%d events processed\n', nev);
65             end
66             if nev >= obj.nevmax
67                 break;
68             end
69         else
70             break;
71         end
72     end
73
74     % call stop() on each Analysis
75     for ia = 1:na
76         ca{ia}.stop();
77     end
78
79     fprintf(1, 'End analysis:  %d events processed\n', nev);
80 end
81
82 end
83
84 end

```

B Code for the graphs

Figure 1a:

```

1 ka = KOSAnalysis;
2 loop = Loop('cdf.dat');
3 loop.run(ka)
4 ka.mass.plot();
5 xlabel('mass [GeV/c^2]');
6 ylabel('entries/(2 MeV/c^2)');

```

Figure 1b:

```
1 ka = KOSAnalysis;
2 ka.minlxy = 8;
3 ka.mind0 = 1.2;
4 loop = Loop('cdf.dat');
5 loop.run(ka)
6 ka.mass.plot();
7 xlabel('mass [GeV/c^2]');
8 ylabel('entries/(2 MeV/c^2)');
```

Figure 2:

```
1 la = LambdaAnalysis;
2 loop = Loop('cdf.dat');
3 loop.run(la)
4 la.mass.plot();
5 xlabel('mass [GeV/c^2]');
6 ylabel('entries/(2 MeV/c^2)');
```

Figure 3a:

```
1 kb = KOSAnalysis();
2 kb.minlxy = -999;
3 kb.mind0 = 0;
4 loop = Loop('cdf.dat');
5 loop.run(kb);
6 x = kb.ct.bins();
7 sig = kb.ct.data - kb.ctside.data;
8 err = sqrt(kb.ct.data + kb.ctside.data);
9 [A, cttau] = meshgrid(11:60, 2.1:0.1:7);
10 chisquared = zeros(50);
11 for i = 2:numel(x)
12     chisquared = chisquared + ((sig(i) - (A ./ cttau) .* exp(-x(i) ./
13         cttau)) ./ err(i)) .^ 2;
14 end
15 surf(A, cttau, chisquared)
16 xlabel('A')
17 ylabel('c * mean lifetime [cm]')
18 zlabel('chi squared value')
```

Figure 3b:

```
1 lb = LambdaAnalysis();
2 lb.minlxy = -999;
3 lb.mind0 = 0;
4 loop = Loop('cdf.dat');
5 loop.run(lb);
6 x = lb.ct.bins();
7 sig = lb.ct.data - lb.ctside.data;
8 err = sqrt(lb.ct.data + lb.ctside.data);
9 [A, cttau] = meshgrid(101:150, 5.1:0.1:10);
10 chisquared = zeros(50);
```

```
11 for i = 2:numel(x)
12     chisquared = chisquared + ((sig(i) - (A ./ ctau) .* exp(-x(i) ./
13         ctau)) ./ err(i)) .^ 2;
14 end
15 surf(A,ctau,chisquared)
16 xlabel('A')
17 ylabel('c * mean lifetime [cm]')
18 zlabel('chi squared value')
```